

Lecture 8: Reinforcement learning

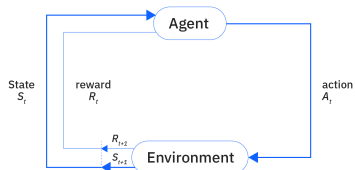
谢丹
清华大学数学系

December 10, 2025

What is Reinforcement Learning?

Definition

Reinforcement Learning (RL) is a machine learning paradigm where an **agent** learns to make decisions by interacting with an **environment** to maximize cumulative **reward**.



Markov Decision Process (MDP)

An MDP is defined by the 5-tuple (S, A, P, R, γ) :

Core Components

- ▶ S : Set of states (state space)
- ▶ A : Set of actions (action space)
- ▶ $P(s'|s, a)$: Transition probability function
- ▶ $R(s, a, s')$: Reward function
- ▶ γ : Discount factor ($0 \leq \gamma \leq 1$)

Markov Property

The future depends only on the current state and action:

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1}|s_t, a_t)$$

Grid World: State Space

State Space S

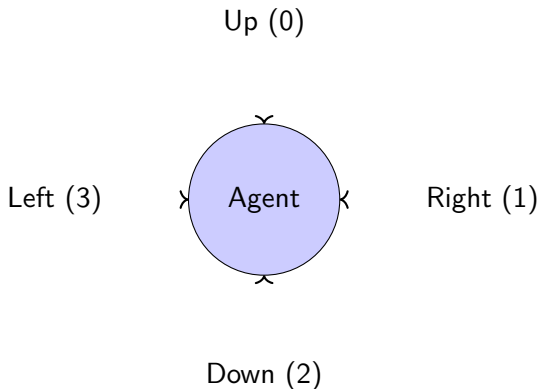
- ▶ 4×4 grid world
- ▶ States represent agent positions
- ▶ $S = \{(x, y) \mid x \in \{0, 1, 2, 3\}, y \in \{0, 1, 2, 3\}\}$
- ▶ $|S| = 16$ states total

(0, 3)	(1, 3)	(2, 3)	Goal
(0, 2)	(1, 2)	(2, 2)	(3, 2)
(0, 1)	(1, 1)	(2, 1)	(3, 1)
Start	(1, 0)	(2, 0)	(3, 0)

Grid World: Action Space

Action Space A

- ▶ Four possible movements
- ▶ $A = \{\text{Up, Right, Down, Left}\}$
- ▶ $A = \{0, 1, 2, 3\}$ (numerical representation)



Transition Function

Transition Probabilities $P(s'|s, a)$

Deterministic transitions with boundary handling:

- ▶ **Up:** $P((x, y - 1)|(x, y), \text{Up}) = 1$ if $y > 0$
- ▶ **Right:** $P((x + 1, y)|(x, y), \text{Right}) = 1$ if $x < 3$
- ▶ **Down:** $P((x, y + 1)|(x, y), \text{Down}) = 1$ if $y < 3$
- ▶ **Left:** $P((x - 1, y)|(x, y), \text{Left}) = 1$ if $x > 0$

Boundary case: If movement is blocked, agent stays in current state.

Terminal State

State $(3, 3)$ is terminal: $P((3, 3)|(3, 3), a) = 1$ for all $a \in A$

Reward Function

Reward Function $R(s, a, s')$

- ▶ **Goal reward:** $R(s, a, s') = +10$ if $s' = (3, 3)$
- ▶ **Step penalty:** $R(s, a, s') = -1$ otherwise

Purpose

- ▶ Positive reward encourages reaching goal
- ▶ Negative reward encourages efficiency (shorter paths)
- ▶ Agent learns to balance exploration vs. exploitation

Note

Reward is sparse: only non-zero when reaching specific states

Discount Factor and Objective

Discount Factor

- ▶ $\gamma = 0.9$
- ▶ Values immediate rewards more than distant ones
- ▶ Ensures infinite sums converge

Agent's goal

Agent's Objective

Maximize the expected discounted return:

$$G_t = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \right]$$

Find optimal policy $\pi^* : S \rightarrow A$ that maximizes G_t from any state.
Notice π is a random choice.

State-Value and Action-Value Functions

State-Value Function $V_{\pi}(s)$

Expected return starting from state s , following policy π :

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

Action-Value Function

$$Q_{\pi}(s, a)$$

Expected return starting from state s , taking action a , then following π :

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

Relationship

$$V_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) Q_{\pi}(s, a)$$

Bellman Equations

Bellman Expectation Equation

$$V^{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [r + \gamma V^{\pi}(s')]$$

$$Q^{\pi}(s, a) = \sum_{s', r} p(s', r|s, a) \left[r + \gamma \sum_{a'} \pi(a'|s') Q_{\pi}(s', a') \right]$$

Bellman Optimality Equations

Optimal state-value:

$$V_*(s) = \max_a \sum_{s', r} P(s', r | s, a) [r + \gamma V_*(s')]$$

Optimal action-value:

$$Q_*(s, a) = \sum_{s', r} P(s', r | s, a) \left[r + \gamma \max_{a'} Q_*(s', a') \right]$$

Optimal policy:

$$\pi_*(s) = \arg \max_a Q_*(s, a) = \arg \max_a \sum_{s', r} P(s', r | s, a) [r + \gamma V_*(s')]$$

Deriving the Bellman Equation for Q-Values

- ▶ Starting point: $q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$
- ▶ Decompose return: $G_t = R_{t+1} + \gamma G_{t+1}$
- ▶ Take expectation:

$$\begin{aligned} q_{\pi}(s, a) &= \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a] \\ &= \mathbb{E}[R_{t+1} \mid s, a] + \gamma \mathbb{E}_{\pi}[G_{t+1} \mid s, a] \end{aligned}$$

Complete Derivation

Step 1: Immediate reward

$$\mathbb{E}[R_{t+1} \mid s, a] = \sum_{s', r} r \cdot p(s', r \mid s, a)$$

Step 2: Future returns

$$\begin{aligned}\mathbb{E}_{\pi}[G_{t+1} \mid s, a] &= \sum_{s'} p(s' \mid s, a) \cdot \mathbb{E}_{\pi}[G_{t+1} \mid S_{t+1} = s'] \\ &= \sum_{s'} p(s' \mid s, a) \cdot v_{\pi}(s')\end{aligned}$$

Step 3: Replace $v_{\pi}(s')$

$$v_{\pi}(s') = \sum_{a'} \pi(a' \mid s') q_{\pi}(s', a')$$

Final Bellman Expectation Equation

Bellman Expectation Equation for Q-Values

$$q_{\pi}(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') q_{\pi}(s', a') \right]$$

In words:

Q-value of $(s, a) =$

- ▶ Expected immediate reward r
- ▶ Plus discounted expected Q-value of next state-action pair
- ▶ Where actions at next state follow policy π

Solution Methods

Dynamic Programming

- ▶ Value Iteration
- ▶ Policy Iteration
- ▶ Requires model knowledge
(P , R)

Reinforcement Learning

- ▶ Q-Learning (off-policy)
- ▶ SARSA (on-policy)
- ▶ Model-free learning

Exploration vs Exploitation

The Fundamental Trade-off

- ▶ **Exploration:** Try new actions to discover better rewards
- ▶ **Exploitation:** Choose known good actions to maximize reward

Common strategies:

- ▶ ϵ -greedy: With probability ϵ , explore randomly
- ▶ Softmax: Choose actions based on probability distribution
- ▶ Upper Confidence Bound (UCB): Balance estimated value and uncertainty

Section 1: Dynamical programming

Dynamic Programming Approach

- ▶ **Perfect Model:** Known $P(s' \mid s, a)$ and $R(s, a, s')$
- ▶ **Markov Property:** Future states depend only on current state and action

Policy Evaluation

Problem: Given π , find v_π

Iterative update:

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) [r + \gamma v_k(s')]$$

As in-place update:

$$v(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) [r + \gamma v(s')]$$

Policy Improvement

Policy Improvement Theorem: If π and π' satisfy:

$$q_{\pi}(s, \pi'(s)) \geq v_{\pi}(s) \quad \forall s \in \mathcal{S}$$

Then:

$$v_{\pi'}(s) \geq v_{\pi}(s) \quad \forall s \in \mathcal{S}$$

Greedy improvement:

$$\pi'(s) = \arg \max_a q_{\pi}(s, a) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]$$

Policy Iteration

Algorithm:

1. Policy Evaluation: v_{π_k}
2. Policy Improvement: $\pi_{k+1}(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi_k}(s')]$

Repeat until $\pi_{k+1} = \pi_k$

Monotonic improvement:

$$v_{\pi_{k+1}}(s) \geq v_{\pi_k}(s) \quad \forall s \in \mathcal{S}$$

Convergence: To π_* in finite iterations for finite MDPs

Value Iteration

Direct optimal value computation:

$$v_{k+1}(s) = \max_a \sum_{s',r} p(s', r|s, a) [r + \gamma v_k(s')]$$

As Bellman optimality operator:

$$v_{k+1} = T^* v_k$$

where T^* is the Bellman optimality operator

Convergence:

$$\lim_{k \rightarrow \infty} v_k = v_*$$

and

$$\|v_k - v_*\|_\infty \leq \gamma^k \|v_0 - v_*\|_\infty$$

Extract optimal policy:

$$\pi_*(s) = \arg \max_a \sum_{s',r} p(s', r|s, a) [r + \gamma v_*(s')]$$

Q-value Dynamic Programming

Q-value Policy Evaluation:

$$q_{k+1}(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') q_k(s', a') \right]$$

Q-value Value Iteration:

$$q_{k+1}(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \max_{a'} q_k(s', a') \right]$$

Mathematical Core of DP in RL

1. **MDP** = $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$
2. **Bellman Equations:**

$$v_{\pi}(s) = \mathbb{E}_{\pi}[r + \gamma v_{\pi}(s')]$$

$$v_{*}(s) = \max_a \mathbb{E}[r + \gamma v_{*}(s')]$$

3. **DP Algorithms:**

- ▶ Policy Evaluation: $v_{k+1} = T^{\pi} v_k$
- ▶ Policy Iteration: Evaluate $\rightarrow$ Improve
- ▶ Value Iteration: $v_{k+1} = T^{*} v_k$

4. **Guarantees:**

- ▶ Contraction mappings: $\|Tv - Tv'\|_{\infty} \leq \gamma \|v - v'\|_{\infty}$
- ▶ Convergence to optimal: $\lim_{k \rightarrow \infty} v_k = v_{*}$

Problem Statement: Frozen Yogurt Shop

Business Challenge: Manage daily inventory to maximize profit

Parameters:

- ▶ **State:** Inventory level
 $s \in \{0, 1, 2, 3, 4, 5\}$
- ▶ **Action:** Order quantity
 $a \in \{0, 1, 2, 3, 4, 5\}$
- ▶ **Max capacity:** 5 tubs
- ▶ **Demand:**
 $d \sim \text{Uniform}\{0, 1, 2, 3\}$

Costs & Prices:

- ▶ Selling price: \$20/tub
- ▶ Order cost: \$12/tub
- ▶ Holding cost: \$2/tub
- ▶ Stockout penalty: \$5/tub

Goal: Find optimal ordering policy $\pi^*(s)$

Mathematical Formulation

Transition Dynamics:

$$s' = \max(0, \min(5, s + a - d))$$

Reward Function:

$$r(s, a, d) = 20 \cdot \min(s+a, d) - 12a - 2 \cdot \max(0, s+a-d) - 5 \cdot \max(0, d-s-a)$$

Demand Distribution:

$$P(d = 0) = 0.1, P(d = 1) = 0.3, P(d = 2) = 0.4, P(d = 3) = 0.2$$

MDP Parameters:

$$\gamma = 0.9, \quad \theta = 0.001$$

Dynamic Programming: Value Iteration

Bellman Optimality Equation:

$$V_{k+1}(s) = \max_a \sum_{d=0}^3 P(d) [r(s, a, d) + 0.9V_k(s')]$$

where $s' = \max(0, \min(5, s + a - d))$

Algorithm Steps:

1. Initialize $V_0(s) = 0$ for all s
2. Repeat until convergence:

$$V_{k+1}(s) = \max_a \mathbb{E}_{d \sim P} [r(s, a, d) + 0.9V_k(s')]$$

3. Extract optimal policy:

$$\pi^*(s) = \arg \max_a \mathbb{E}_{d \sim P} [r(s, a, d) + 0.9V_*(s')]$$

Manual Calculation: State $s=2$, First Iteration

Compute $V_1(2)$:

Action a	Value	Breakdown
$a = 0$	28.0	$0.1 \times (-4) + 0.3 \times 18 + 0.4 \times 40 + 0.2 \times 35$
$a = 1$	14.4	$0.1 \times (-18) + 0.3 \times 4 + 0.4 \times 26 + 0.2 \times 23$
$a = 2$	0.4	$0.1 \times (-32) + 0.3 \times (-10) + 0.4 \times 12 + 0.2 \times 9$
$a \geq 3$	< 0	Negative (exceeds capacity or high holding costs)

Best action: $a = 0$ with value 28.0

$$V_1(2) = 28.0, \quad \pi_1(2) = 0$$

First Iteration Results

State s	Optimal a	$V_1(s)$
0	3	16.8
1	2	24.6
2	0	28.0
3	0	24.6
4	0	16.8
5	0	0.0

Initial Insights:

- ▶ Order when inventory is low (states 0,1)
- ▶ Don't order when inventory is high (states 2-5)
- ▶ Value peaks at intermediate inventory levels

Second Iteration: State $s=2$

Using V_1 from previous iteration:

$$V_2(2) = \max_a \mathbb{E}[r + 0.9V_1(s')]$$

$$\begin{aligned} a = 0 : & 0.1(-4 + 0.9 \times 28.0) + 0.3(18 + 0.9 \times 24.6) \\ & + 0.4(40 + 0.9 \times 16.8) + 0.2(35 + 0.9 \times 16.8) \\ & = 2.12 + 12.04 + 22.05 + 10.02 = 46.23 \end{aligned}$$

$$\begin{aligned} a = 1 : & 0.1(-18 + 0.9 \times 24.6) + 0.3(4 + 0.9 \times 28.0) \\ & + 0.4(26 + 0.9 \times 24.6) + 0.2(23 + 0.9 \times 16.8) \\ & = 0.41 + 8.76 + 19.26 + 7.62 = 36.05 \end{aligned}$$

Update: $V_2(2) = 46.23$, still $a = 0$

Convergence Pattern

Value Iteration Progress

Iteration	$V(0)$	$V(2)$	$V(5)$
0	0.00	0.00	0.00
1	16.80	28.00	0.00
2	46.23	46.23	16.80
5	102.35	112.48	92.35
10	132.18	144.21	128.15
20	137.89	149.92	137.89
45	138.42	150.28	138.42

Converges after 45 iterations ($\Delta < 0.001$)

Optimal Solution

Final Optimal Values and Policy

State s	$V^*(s)$	$\pi^*(s)$
0	138.42	3
1	146.35	2
2	150.28	1
3	150.28	0
4	146.35	0
5	138.42	0

Optimal Policy Interpretation:

$$\pi^*(s) = \max(0, 3 - s)$$

"Order up to 3 tubs" policy

Section 2: Q-learning

Q-Learning: Iterative Updates

In Q-Learning, we optimize the Q-table by iterative updates ($t = 0, 1, 2, \dots$).

In the equation below:

- ▶ $Q_t(s_t, a_t)$ is the current Q-value
- ▶ $Q_{t+1}(s_t, a_t)$ is the updated Q-value

Q-Learning Update Equation

Core Update Rule

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha \left(r_t + \gamma \max_a Q_t(s_{t+1}, a) - Q_t(s_t, a_t) \right)$$

where α is the learning rate. The current state is s_t , action is a_t , the reward is r_t , and the next state is s_{t+1} . Q_t is the current Q table.

Understanding the Equation: Step 1

Suppose you executed an action a_t on state s_t , and as a result:

- ▶ You got reward r_t
- ▶ The state changed to s_{t+1}

$$\begin{array}{ccc} \text{State } s_t & \xrightarrow{\text{Action } a_t} & \text{State } s_{t+1} \\ & \text{Reward: } r_t & \end{array}$$

Understanding the Equation: Step 2

The optimal next action at s_{t+1} is:

$$a_{t+1} = \arg \max_a Q(s_{t+1}, a)$$

By taking this optimal action, you get the expected future discounted reward:

$$r_t + \gamma \max_a Q(s_{t+1}, a)$$

Where:

- ▶ r_t : Immediate reward
- ▶ $\gamma \max_a Q(s_{t+1}, a)$: Maximum discounted future reward

Understanding the Equation: Step 3

1. Calculate the **TD target** (optimal expected value):

$$\text{Target} = r_t + \gamma \max_a Q_t(s_{t+1}, a)$$

2. Compare with **current estimate**:

$$\text{TD Error} = \left(r_t + \gamma \max_a Q_t(s_{t+1}, a) \right) - Q_t(s_t, a_t)$$

3. Update current value using learning rate α :

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha \times \text{TD Error}$$

Q-Learning Update: Summary

$$\underbrace{Q_{t+1}(s_t, a_t)}_{\text{New value}} = \underbrace{Q_t(s_t, a_t)}_{\text{Current value}} + \alpha \left(\underbrace{r_t + \gamma \max_a Q_t(s_{t+1}, a)}_{\text{Target}} - \underbrace{Q_t(s_t, a_t)}_{\text{Current estimate}} \right)$$

Key Components

- ▶ α : Learning rate (how much to update)
- ▶ γ : Discount factor (importance of future rewards)
- ▶ $\max_a Q_t(s_{t+1}, a)$: Best estimated future value

The Exploration-Exploitation Dilemma

Fundamental Challenge in RL

- ▶ **Exploitation:** Choose the action with the highest known reward
- ▶ **Exploration:** Try new actions to discover potentially better rewards

Why is this hard?

Too much exploitation → Might miss better options

Too much exploration → Never converges to optimal policy

Epsilon-Greedy Strategy: The Solution

Simple yet effective approach

- ▶ ϵ : A small probability (e.g., 0.1 or 0.01)
- ▶ With probability $1 - \epsilon$: Choose **greedy** action (exploit)
- ▶ With probability ϵ : Choose **random** action (explore)

Algorithm

Input: Current state s , exploration rate ϵ , Q-table Q

1. Generate random number $r \sim U(0, 1)$
2. If $r < \epsilon$:
 - ▶ Choose random action a uniformly
3. Else:
 - ▶ Choose greedy action $a = \arg \max_{a'} Q(s, a')$

What is Deep Q-Network (DQN)?

Core Concept

Deep Q-Network (DQN) is a **deep reinforcement learning method** that combines:

- ▶ **Q-Learning** principles
- ▶ **Neural networks** (deep learning)

Key Motivation

The idea is strongly motivated by the plain Q-learning method discussed previously.

Why move from Q-tables to neural networks?

- ▶ Better handling of high-dimensional state spaces
- ▶ More efficient representation of complex environments
- ▶ Ability to generalize across similar states

Limitations of Q-Learning with Tables

The Problem with Q-Tables

- ▶ Discretizing states in a Q-table is **not computationally efficient**
- ▶ It's **not smart** for complex environments
- ▶ Requires storing a separate value for **every possible state-action pair**
- ▶ **Cannot generalize** from seen states to unseen states

The "Curse of Dimensionality"

- ▶ For environments with many states, Q-tables become **huge**
- ▶ Memory requirements grow **exponentially** with state dimensions
- ▶ Learning becomes **impractical** for real-world problems

The DQN Solution: From Table to Network

Core Idea

Instead of using a Q-table, DQN uses:

- ▶ A **Q-function** $Q(s, a; \theta)$
- ▶ Implemented as a **neural network** (Q-network)
- ▶ Parameterized by weights θ

Network Structure

- ▶ **Input:** State representation
- ▶ **Output:** Q-values for each action
- ▶ **Hidden layers:** Learn complex features
- ▶ Can handle **continuous** state spaces

How DQN Works: The Basic Approach

Similarity to Q-Learning

- ▶ Like in Q-learning, we optimize the Q-network to match actual experiences
- ▶ Goal: Learn optimal Q-values $Q^*(s, a)$
- ▶ Use the same TD-learning principle

Loss Function for Q-Network

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

Where:

- ▶ θ : Current network parameters
- ▶ θ^- : Target network parameters
- ▶ $\mathcal{D}$: Experience replay buffer

The Challenge: Training Stability

Why is Deep Q-Learning Difficult?

- ▶ Neural networks are **non-linear** function approximators
- ▶ Sequential observations are **correlated**
- ▶ Q-value estimates can become **unstable**
- ▶ The target values **change** during training

Key Questions

1. How do we improve training stability?
2. What tricks can help DQN learn effectively?
3. How are these implemented in practice?

DQN's Key Innovations

Three Crucial Techniques

DQN introduced several innovations to stabilize training:

1. Experience Replay

- ▶ Store experiences in a buffer
- ▶ Sample random batches during training
- ▶ Breaks correlation between consecutive samples

2. Target Network

- ▶ Separate network for target Q-values
- ▶ Updated periodically
- ▶ Stabilizes the learning target

3. Gradient Clipping

- ▶ Limits the size of gradient updates
- ▶ Prevents exploding gradients

Section 3: Policy-based method

Policy-Based Methods: Core Concept

Fundamental Idea

In policy-based methods, we **directly optimize** the policy $\pi(a|s)$ itself, bypassing the need to learn value functions first.

- ▶ We parameterize the policy as $\pi_{\theta}(a|s)$ with parameters θ
- ▶ The objective is to find the optimal parameters:

$$\theta^* = \arg \max_{\theta} J(\theta)$$

where $J(\theta)$ is the expected return under policy π_{θ}

Why Parameterize?

- ▶ **Generalization:** Enables handling of unseen states
- ▶ **Compact representation:** Avoids storing policies for each state individually
- ▶ **Gradient optimization:** Allows use of gradient ascent methods

Mathematical Framework

Objective

Maximize expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)]$$

Policy Gradient Theorem

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t \right]$$

where $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$, τ is trajectory generated from π_{θ} .

Monte Carlo Estimation

From Episode Samples

For episode $\tau = (s_0, a_0, r_0, \dots, s_T)$:

$$g(\tau) = \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t$$

Batch Average

With N episodes:

$$\bar{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) \cdot G_t^{(i)}$$

Step 1: Compute Returns

Discounted Return from Time t

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots + \gamma^{T-t} r_T$$

Recursive Computation

$$G_T = r_T$$

$$G_t = r_t + \gamma G_{t+1} \quad \text{for } t = T-1, \dots, 0$$

Example

For $\gamma = 0.9$, $T = 3$:

$$G_1 = r_1 + 0.9r_2 + 0.9^2r_3$$

Step 2: Log-Probability Gradients

Discrete Actions (Softmax)

$$\pi_{\theta}(a|s) = \frac{e^{\theta^{\top} \phi(s,a)}}{\sum_{a'} e^{\theta^{\top} \phi(s,a')}}$$

$$\nabla_{\theta} \log \pi_{\theta}(a|s) = \phi(s, a) - \sum_{a'} \pi_{\theta}(a'|s) \phi(s, a')$$

Continuous Actions (Gaussian)

$$\pi_{\theta}(a|s) = \mathcal{N}(\mu_{\theta}(s), \sigma^2)$$

$$\nabla_{\theta} \log \pi_{\theta}(a|s) = \frac{a - \mu_{\theta}(s)}{\sigma^2} \nabla_{\theta} \mu_{\theta}(s)$$

Step 3: Gradient Accumulation

Algorithm

$$g \leftarrow 0$$

for $t = 0$ to T :

$$g \leftarrow g + \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t$$

Parameter Update

$$\theta \leftarrow \theta + \alpha \cdot g$$

where α is the learning rate

Complete REINFORCE Algorithm

1. Initialize policy parameters θ
2. For iteration $k = 1, 2, \dots$:
 - ▶ Collect N episodes using π_θ
 - ▶ Compute gradient estimate g
 - ▶ Update $\theta \leftarrow \theta + \alpha g$
3. Until convergence

Gradient theorem: Proof

State Distribution Form

$$J(\theta) = \mathbb{E}_{s_0 \sim p(s_0)} [V^{\pi_\theta}(s_0)] = \sum_s p(s_0 = s) V^{\pi_\theta}(s)$$

State-Value Function

$$V^{\pi_\theta}(s) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \middle| s_0 = s \right]$$

Step 1: Bellman Equations

Value Function

$$V^{\pi_{\theta}}(s) = \sum_a \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a)$$

Action-Value Function

$$Q^{\pi_{\theta}}(s, a) = r(s, a) + \gamma \sum_{s'} p(s'|s, a) V^{\pi_{\theta}}(s')$$

Step 2: Gradient of Value Function

Take gradient with respect to θ :

$$\nabla_{\theta} V^{\pi_{\theta}}(s) = \nabla_{\theta} \left[\sum_a \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a) \right]$$

Apply product rule:

$$\nabla_{\theta} V^{\pi_{\theta}}(s) = \sum_a \left[\underbrace{\nabla_{\theta} \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a)}_{\text{Policy gradient}} + \underbrace{\pi_{\theta}(a|s) \nabla_{\theta} Q^{\pi_{\theta}}(s, a)}_{\text{Value gradient}} \right]$$

Step 3: Expand $\nabla_{\theta} Q^{\pi_{\theta}}(s, a)$

From Bellman equation:

$$Q^{\pi_{\theta}}(s, a) = r(s, a) + \gamma \sum_{s'} p(s'|s, a) V^{\pi_{\theta}}(s')$$

Since $r(s, a)$ and $p(s'|s, a)$ don't depend on θ :

$$\nabla_{\theta} Q^{\pi_{\theta}}(s, a) = \gamma \sum_{s'} p(s'|s, a) \nabla_{\theta} V^{\pi_{\theta}}(s')$$

Step 4: Recursive Formulation

Substitute back:

$$\begin{aligned}\nabla_{\theta} V^{\pi_{\theta}}(s) &= \sum_a \nabla_{\theta} \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a) \\ &\quad + \gamma \sum_a \pi_{\theta}(a|s) \sum_{s'} p(s'|s, a) \nabla_{\theta} V^{\pi_{\theta}}(s')\end{aligned}$$

Define transition probability:

$$P(s \rightarrow s', 1, \pi_{\theta}) = \sum_a \pi_{\theta}(a|s) p(s'|s, a)$$

$$\nabla_{\theta} V^{\pi_{\theta}}(s) = \sum_a \nabla_{\theta} \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a) + \gamma \sum_{s'} P(s \rightarrow s', 1, \pi_{\theta}) \nabla_{\theta} V^{\pi_{\theta}}(s')$$

Step 5: Unroll Recursion

This is a recursive equation. Unroll infinitely:

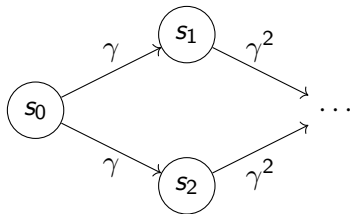
$$\nabla_{\theta} V^{\pi_{\theta}}(s) = \sum_{x \in \mathcal{S}} \sum_{k=0}^{\infty} \gamma^k P(s \rightarrow x, k, \pi_{\theta}) \sum_a \nabla_{\theta} \pi_{\theta}(a|x) Q^{\pi_{\theta}}(x, a)$$

where $P(s \rightarrow x, k, \pi_{\theta})$ is the probability of reaching state x from s in k steps under π_{θ} .

Step 6: Discounted State Distribution

Define discounted state visitation distribution:

$$d^{\pi_{\theta}}(s) = \sum_{s_0} p(s_0) \sum_{k=0}^{\infty} \gamma^k P(s_0 \rightarrow s, k, \pi_{\theta})$$



Step 7: Gradient of Objective Function

Recall $J(\theta) = \sum_s p(s_0 = s) V^{\pi_\theta}(s)$

$$\nabla_\theta J(\theta) = \sum_s p(s_0 = s) \nabla_\theta V^{\pi_\theta}(s)$$

Substitute from Step 5:

$$\nabla_\theta J(\theta) = \sum_s p(s_0 = s) \sum_x \sum_{k=0}^{\infty} \gamma^k P(s \rightarrow x, k, \pi_\theta) \sum_a \nabla_\theta \pi_\theta(a|x) Q^{\pi_\theta}(x, a)$$

Step 8: Rearrange Summations

Change order of summations:

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \sum_x \left[\sum_s p(s_0 = s) \sum_{k=0}^{\infty} \gamma^k P(s \rightarrow x, k, \pi_{\theta}) \right] \\ &\quad \times \sum_a \nabla_{\theta} \pi_{\theta}(a|x) Q^{\pi_{\theta}}(x, a)\end{aligned}$$

The term in brackets is $d^{\pi_{\theta}}(x)$:

$$\nabla_{\theta} J(\theta) = \sum_x d^{\pi_{\theta}}(x) \sum_a \nabla_{\theta} \pi_{\theta}(a|x) Q^{\pi_{\theta}}(x, a)$$

Step 9: Log-Derivative Trick

Use identity:

$$\nabla_{\theta} \pi_{\theta}(a|s) = \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s)$$

Substitute:

$$\nabla_{\theta} J(\theta) = \sum_s d^{\pi_{\theta}}(s) \sum_a \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a)$$

Rewrite as expectation:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s \sim d^{\pi_{\theta}}, a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a)]$$

Step 10: Trajectory-Based Form

Alternative expression using complete trajectories:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{\infty} \gamma^t Q^{\pi_{\theta}}(s_t, a_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

Or using returns $G_t = \sum_{k=t}^{\infty} \gamma^{k-t} r(s_k, a_k)$:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t \right]$$

The Policy Gradient Theorem

Theorem 1 (State-Action Form)

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s \sim d^{\pi_{\theta}}, a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a)]$$

Theorem 2 (Trajectory Form)

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) G_t \right]$$

Key Insights

No Model Needed

- ▶ Only requires sampling
- ▶ No environment dynamics
- ▶ Model-free approach

Credit Assignment

- ▶ Each action adjusted by contribution
- ▶ Weighted by Q-value
- ▶ Temporal credit assignment

Log-Derivative Trick

- ▶ Converts probability ratio
- ▶ Enables Monte Carlo estimation
- ▶ Simple gradient computation

Compatible Approximation

- ▶ Can use $Q_w(s, a)$ approximation
- ▶ Under certain conditions
- ▶ No bias introduced

Why This Matters

Theoretical Foundation

- ▶ Justifies REINFORCE algorithm
- ▶ Basis for actor-critic methods
- ▶ Connects policy and value methods

Practical Implications

- ▶ Enables direct policy optimization
- ▶ Handles continuous action spaces
- ▶ Foundation for modern RL algorithms

From Theorem to Algorithm

1. Sample trajectories: $\tau \sim \pi_\theta$
2. Compute returns: $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$
3. Estimate gradient: $\hat{g} = \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) G_t$
4. Update parameters: $\theta \leftarrow \theta + \alpha \hat{g}$

This gives us the REINFORCE algorithm!

Extension: Advantage Function

Replace $Q^{\pi_\theta}(s, a)$ with advantage $A^{\pi_\theta}(s, a)$:

$$A^{\pi_\theta}(s, a) = Q^{\pi_\theta}(s, a) - V^{\pi_\theta}(s)$$

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim d^{\pi_\theta}, a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a|s) A^{\pi_\theta}(s, a)]$$

Lower variance, same expectation!

Summary

- ▶ Derived Policy Gradient Theorem from first principles
- ▶ Showed connection between policy gradient and value functions
- ▶ Established theoretical foundation for policy optimization
- ▶ Demonstrated practical algorithm derivation
- ▶ Provided basis for modern RL methods

Policy Gradient: Theory to Practice

Advantage Function

Actor-Critic Approach

Replace $Q^\pi(s, a)$ with a learned estimate:

$$\nabla_\theta J(\theta) \approx \mathbb{E} \left[\nabla_\theta \log \pi(a|s; \theta) \cdot \hat{Q}(s, a; \phi) \right]$$

where $\hat{Q}(s, a; \phi)$ is the critic's estimate.

Definition

Advantage function measures how much better an action is compared to average:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

Actor-Critic with Advantage

Update rule becomes:

$$\nabla_\theta J(\theta) \approx \mathbb{E} \left[\nabla_\theta \log \pi(a|s; \theta) \cdot A^\pi(s, a) \right]$$

Types of Critics

Value-Based Critic

- ▶ Learns $V(s)$: state-value function
- ▶ Advantage:
 $A(s, a) = Q(s, a) - V(s)$
- ▶ For discrete actions:
 $Q(s, a) = r + \gamma V(s')$
- ▶ Simple but effective

Q-Based Critic

- ▶ Learns $Q(s, a)$:
action-value function
- ▶ Direct advantage:
 $A(s, a) = Q(s, a) - V(s)$
- ▶ Where
 $V(s) = \mathbb{E}_{a \sim \pi}[Q(s, a)]$
- ▶ More expressive but
complex

A2C: Advantage Actor-Critic

Key Features

- ▶ **Synchronous** version of A3C (without asynchrony)
- ▶ Uses n-step returns for advantage estimation
- ▶ Multiple parallel environments
- ▶ More stable than one-step AC

Advantage Estimation

$$A(s_t, a_t) = \sum_{i=0}^{k-1} \gamma^i r_{t+i} + \gamma^k V(s_{t+k}) - V(s_t)$$

where k is the number of steps.

Update Rules

$$\theta \leftarrow \theta + \alpha_{\theta} \nabla_{\theta} \log \pi(a|s; \theta) A(s, a)$$

PPO: Proximal Policy Optimization

Motivation

Prevent too large policy updates that could collapse performance.

Clipped Objective

$$J^{CLIP}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

where $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the probability ratio.

- ▶ **Clipping:** Prevents large updates
- ▶ **Multiple epochs:** Reuse experience
- ▶ **GAE:** Generalized Advantage Estimation
- ▶ State-of-the-art for many tasks

SAC: Soft Actor-Critic

Key Idea

Maximum entropy reinforcement learning: encourage exploration.

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_t r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t)) \right]$$

where $\mathcal{H}$ is entropy, α is temperature.

Components

- ▶ **Stochastic actor:** $\pi(a|s)$
- ▶ **Two Q-networks:** $Q_{\theta_1}, Q_{\theta_2}$
- ▶ **Value network:** V_{ψ}
- ▶ **Target networks:** Stable learning

Advantages

- ▶ State-of-the-art for continuous control
- ▶ Automatic temperature adjustment
- ▶ Very sample efficient
- ▶ Robust to hyperparameters